#6

CS 0447
Introduction to
Computer Programming

Flow, Conditionals, and Loops

Luís Oliveira

Original slides by: Jarrett Billingsley
Modified with bits from: Bruce Childers, David Wilkinson

Fall 2020

So far…

- Putting numbers into registers

```
li a0, 3
```

```
la a0, x
```

label

```
.data
x: .word 4
```

- COPYing register contents

```
move a0, t0
```

Unsigned! ➔ These do zero extension

- COPYing from/to memory

lw/sw, lh/lhu/sh, lb/lbu/sb

```
la t1, x
lw t0, 0(t1)
```

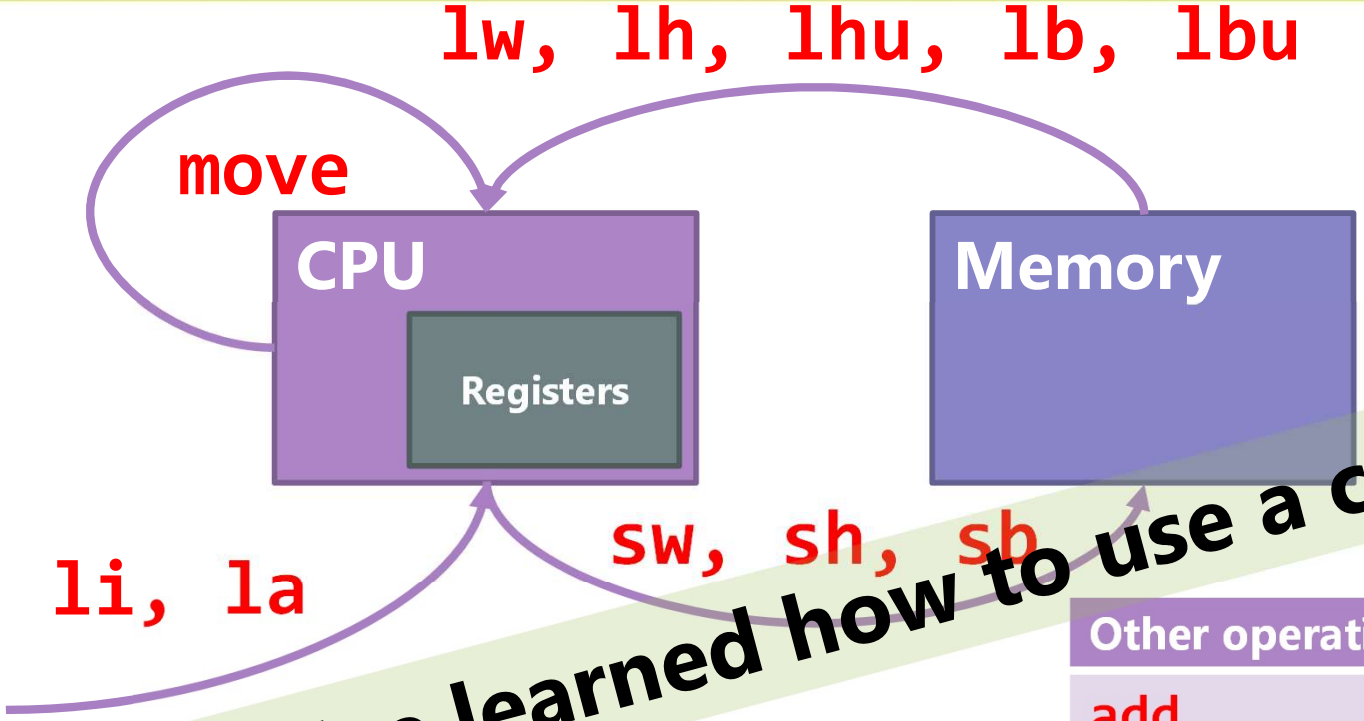
```
lw t0, x
```

```
la t1, x
sw t0, 0(t1)
```

```
sw t0, x
```

Do the same thing

In another perspective



We've learned how to use a calculator!

Other operations
add
sub
mul
syscall
…

Datatypes
word
half
byte
asciiz
…

Introduction to conditions

- What distinguishes a computer from a calculator?
- It can make **decisions** based on values that it calculates
  - If the value of this register is this, do something.
  - Otherwise, do something else.
- The possible decisions make up the *potential* **control flow** of the program.
  - When there is no possible route to a piece of code in your program, that is called **dead code**.
    - It's like procrastination!

```
if(false) {
    do_some_work()
}
```

Control flow

With great power…

- **Control flow** determines the order that your instructions run in
  - What kinds of control flow statements do you know of?
  - What about functions?

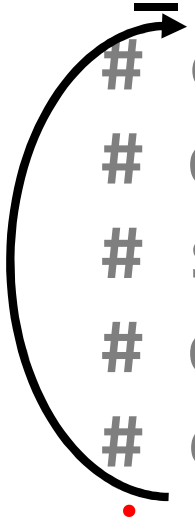
- **In asm, the only thing you get for free is that instructions run in order**

- **You're responsible for coming up with everything else.**
  - If you screw up your control flow, the CPU doesn't care
  - You'll just have a broken, malfunctioning program
    - And it'll be half an hour before the lab is due
      - And you'll be sad
        - This is like 90% of the bugs

Getting a little further from familiarity

- all control flow is done with **branches** and **jumps**
  - these are instructions which say "go somewhere else"
- for example…

```
_main_loop:
    # clear screen
    # draw one thing
    # sleep
    # draw another thing
    # etc
    j _main_loop
```



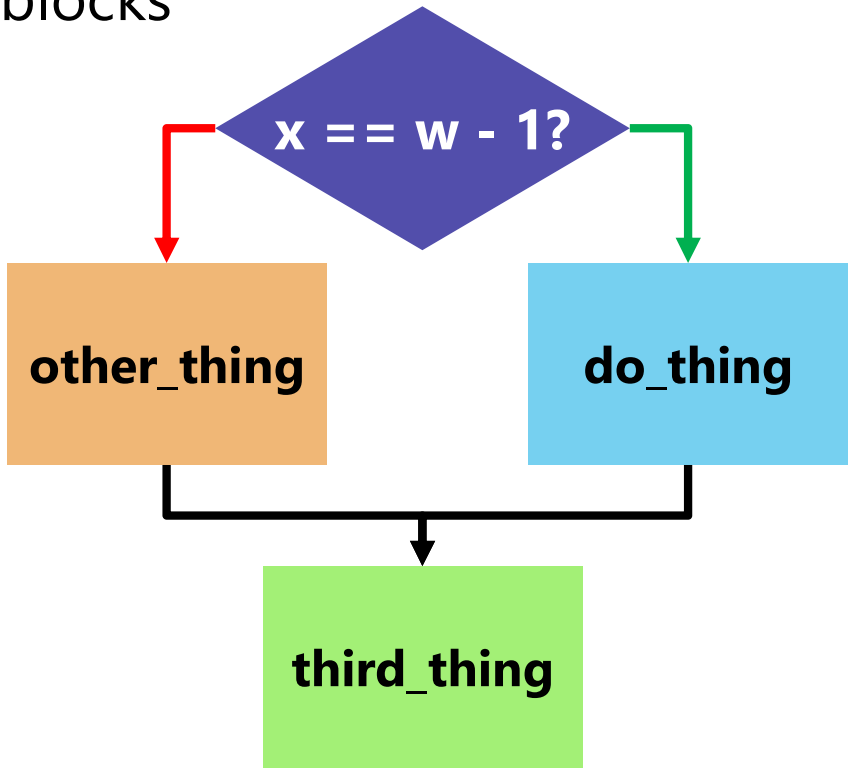
this is an infinite loop, which is sometimes useful but not too interesting

j stands for "jump" – go somewhere else

Building blocks

- A **basic block** is a chunk of code that has no control flow in it
- Control flow statements separate basic blocks

```
if(x == w - 1) {
    do_thing()
} else {
    other_thing()
}
third_thing()
```



thinking about this is REAL HELPFUL

Essentially…

- The way control flow works in asm is **you make basic blocks**
  - You gotta **name (label) them**

- Then, you use special instructions to choose where to go
  - **Ask yourself** "Which basic block runs next?"
  - *Select the instruction you need!*
    - *Don't worry, we look into these instructions in a moment*

- ***And don't forget!***
  - *Write pseudo-code (with comments) to keep track of control flow*
  - *Or make a drawing of a flow-chart!*
  - *Or … any other guide you think it's helpful*

Conditionals: if and if-else

MIPS ISA: conditional branch instructions

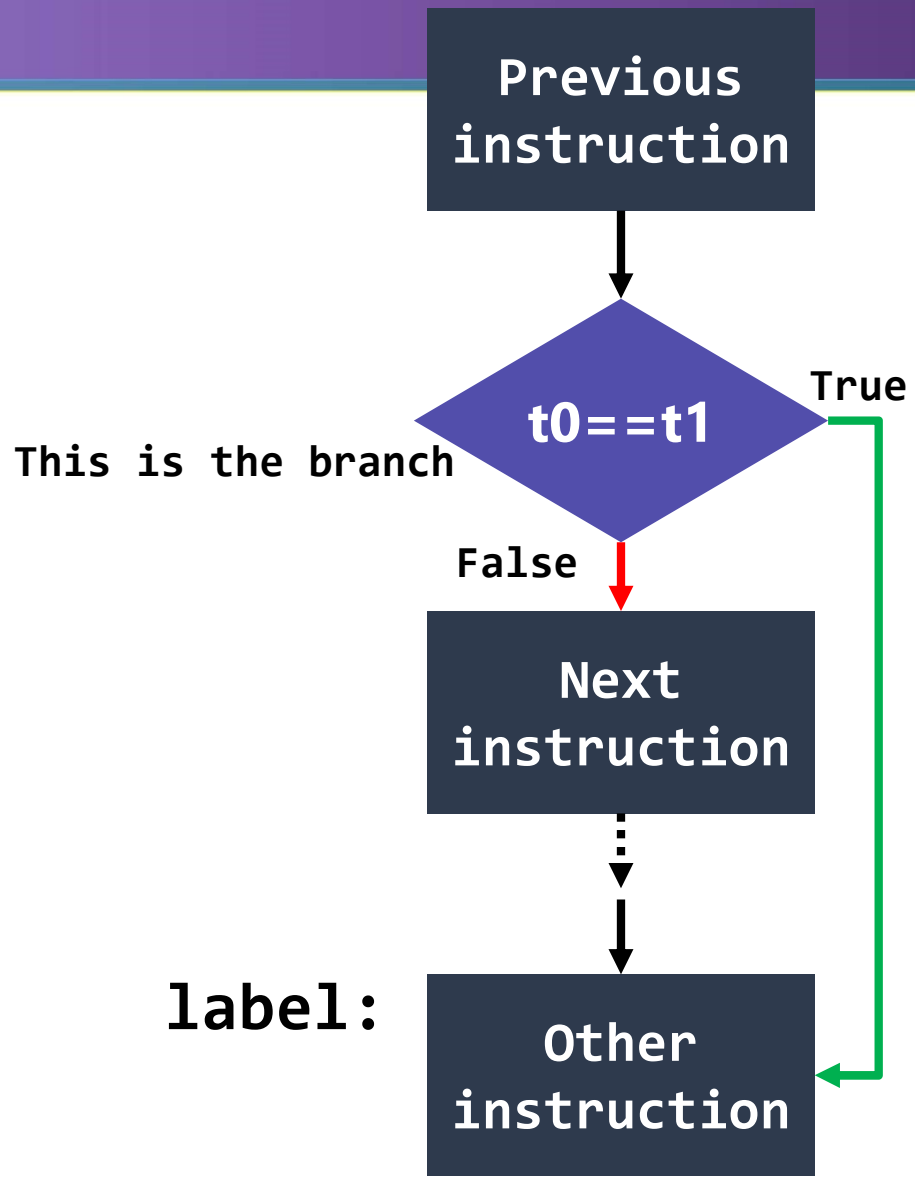
- **conditional** branch instructions do one of two things:
  - if the condition is met, we go to the label
  - otherwise, **nothing happens,** and we go to the **next instruction**

Instruction	Meaning
`beq a, b, label`	`if(a == b) { goto label }`
`bne a, b, label`	`if(a != b) { goto label }`

above, **a** must be a register, but **b** can be a register or immediate
(by the powers of the pseudo-instruction)

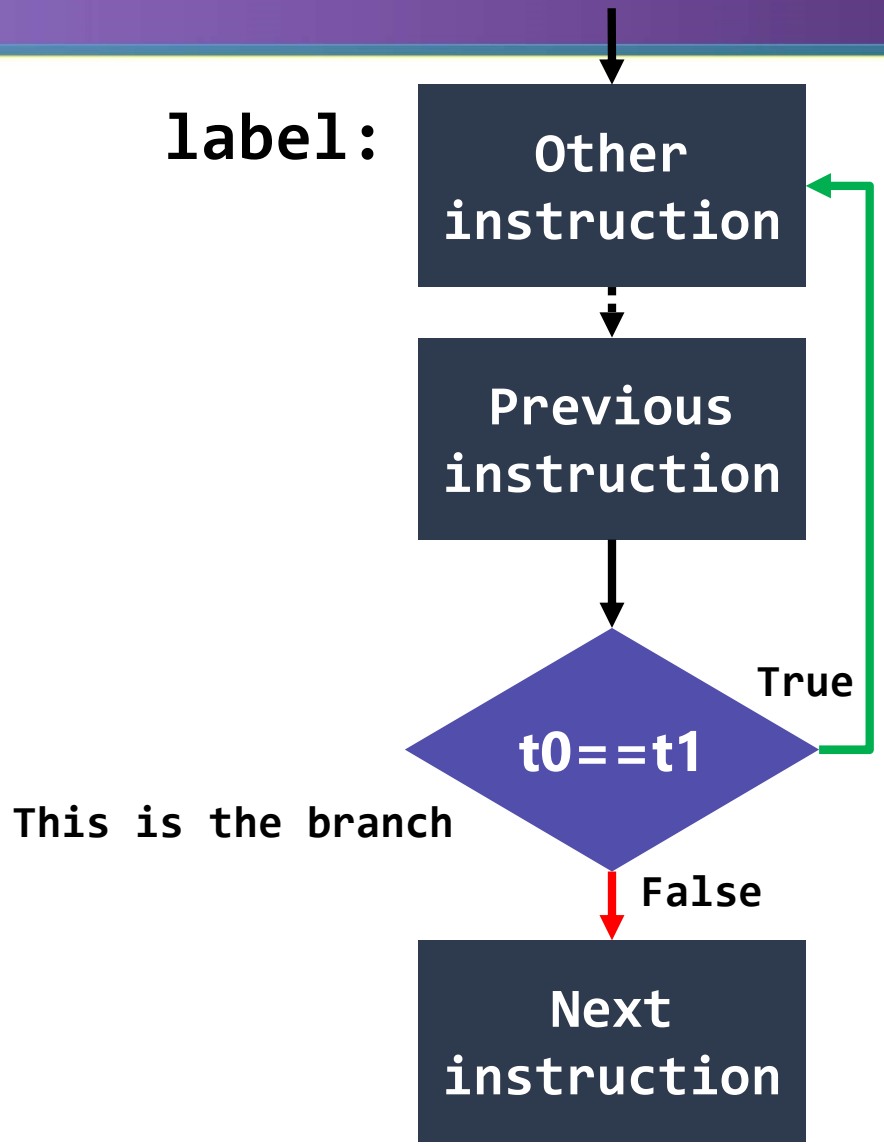
How do these work?

```
beq t0, t1, label
# branch if equal
```



How do these work?

```
beq t0, t1, label
# branch if equal
```



How to write asm (again!)

- Remember: **WRITE PSEUDOCODE ALWAYS REALLY!!!**

```
if(x == w - 1) {

} else {

}
```

IGNORE THE STUFF INSIDE

TRANSLATE THE CONTROL FLOW TO ASM *FIRST!!!!!!*

Like *mad libs*, but for code

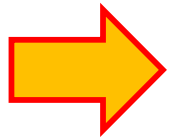
- From now on, I'll use these 'blocks' to represent the basic blocks
  - cause they don't matter

```
if(some condition) {
        block A
} else {
        block B
}
    block C
```

A simple conditional block (if)

- If there is no *else*, it's pretty simple.

```
if(s0 == 30) {
        block A
}
   block B
```



```
bne s0, 30, blockB
blockA:
```

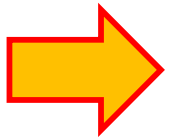
Wait, what???? Isn't that Branch NOT equal?

O.O

A simple conditional block (if)

- If there is no *else*, it's pretty simple.

```
if(s0 == 30) {
      block A
}
   block B
```



```
bne s0, 30, blockB
```

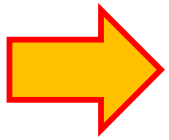
In Java/C what happens in an if?
You JUMP OVER when the condition is true or false?

When its FALSE!!

A simple conditional block (if)

- In MIPS you jump when the condition is TRUE

```
if(s0 == 30) {
        block A
}
   block B
```



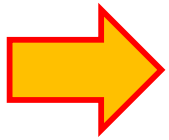
```
    bne s0, 30, blockB
blockA:
        block A
blockB:
        block B
```

coming up with unique names for all the control flow labels is kind of a chore

An if-else with a simple condition

- more blocks now…

```
if(s0 == 30) {
        block A
}
else {
        block B
}
   block C
```



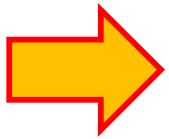
```
bne s0, 30, blockB
   block A
   j blockC
blockB:
   block B
blockC:
   block C
```

we **NEED THIS** – the CPU doesn't see/care about your labels!!

The other way around

- Because in HLL we “execute smth if” and In assembly we “jump over if”
- We usually **negate** the condition in the assembly to skip over code
  - It’s a preference.
  - You can still invert the process
    - How?

```
if(s0 == 30) {
        block A
}
else {
        block B
}
    block C
```



```
    beq s0, 30, blockA
    j blockElse
blockA:
        block A
    j blockExit # skip the else
blockElse:
        block B
blockExit:
        block C
```

MIPS ISA: conditional branch instructions

- MIPS also supports instructions that compare to zero

Instruction	Meaning
`bltz a, label`	`if(a < 0) { goto label }`
`blez a, label`	`if(a <= 0) { goto label }`
`bgtz a, label`	`if(a > 0) { goto label }`
`bgez a, label`	`if(a >= 0) { goto label }`

MIPS ISA: set if less than

- And…

Instruction	Meaning
`slt c, a, b`	`if(a < b) { c = 1 } else { c = 0 }`

Set if Less Than: register c will be set to 1 if a<b.
Otherwise, register c will be set to 0.

Using **`slt`** together with **`bne`** and **`beq`** all conditionals can be implemented!
a=b , a≠b, a>b, a≥b, a<b, a≤b

Thanks, De Morgan



MIPS ISA: conditional branch instructions

- **Or…** we can just use the **pseudo-instructions :D**

Instruction	Meaning
`blt a, b, label`	`if(a < b) { goto label }`
`ble a, b, label`	`if(a <= b) { goto label }`
`bgt a, b, label`	`if(a > b) { goto label }`
`bge a, b, label`	`if(a >= b) { goto label }`

above, **a** must be a register, but **b** can be a register or immediate

Examples

Example 1: branch if **a>b**

```
    bgt a, b, label          # Goto label if a>b
```

Solution: branch if **b<a**

```
    slt t, b, a              # t=1 if b<a
    bne t, zero, label       # Goto label if t≠0
```

Example 2: branch if **a≥b**

```
    bge a, b, label          # Goto label if a≥b
```

Solution: branch if **!(a<b)**

```
    slt t, a, b              # t=1 if a<b
    beq t, zero, label       # Goto label if t=0
```

Complex conditionals

In this code...

```
if(dog_size < 10 || dog_name() == "Fluffy")
```

if dog_size is 3, **is dog_name() called?**

NO!

this is **short circuit evaluation.**

for || (logical OR), if the first condition is true, **the second one is skipped.** (cause there's no way for the result of the OR to be false.)

for && (logical AND), if the first condition is *false*, **the second one is skipped.**

In this code...

```
if(dog_size < 10)
     small();
else if(dog_size < 20)
     medium();
else if(dog_size < 30)
     large();
else
     enormous();
```

if dog_size is 3, **is this condition checked?**

NO!

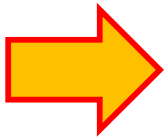
once a true condition is found, **no more conditions are checked.** after small(), it comes down here.

And-and!

- Block A is run if ***both* conditions are true.**
  - to think of it another way… it's *skipped* if? What's the inverse?
  - either condition is *false*…



```
if(s0 == 30 &&
   s1 > 1) {
    block A
}
```



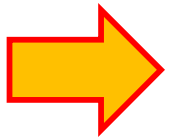
```
bne s0, 30, skipA
ble s1,  1, skipA
    block A
skipA:
```

Or-or!

- We go to block A if ***either*** **condition is true.**
  - to think of it another way… it's *skipped* if? What's the inverse?
  - all conditions are *false*.

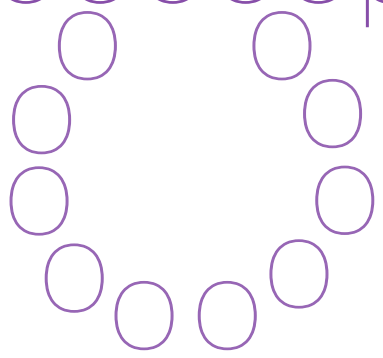


```
if(s0 == 30 ||
   s1 > 1)
{
    block A
}
```



```
    beq s0, 30, blockA
    ble s1,  1, skipA
blockA:
    block A
skipA:
```

Looooops



Dis-assembling a for-loop

- How does a for loop work?

```
for(i=0; i<10; i++)
{
    block A
}
// carry on
```

What is the first thing a for does?

Initialize: i=0

And???

Check condition:
execute while i<10

Then…

block A

Finally?

Increment: i++

Go back up to the top



Looping in MIPS assembly

- Let's use **s0** to hold **i**

```
for(i=0; i<10; i++)
{
    block A
}
// carry on
```

Let's start with a recipe

```
    li s0, 0
loop_top:
    block A
    addi s0, s0, 1
    j loop_top
carry_on:
    # carry on
```

What's the first thing a *for* does?

which conditional branch?

How do we increment?

How do we go up?

That's **bge**, actually

We want to **leave** the loop…
when the **opposite** of i<10
happens!

- In HLL we "execute smth if"
- In assembly we "jump over if"
- Thus **negate** the condition in the assembly to skip over code

```
        li s0, 0
loop_top:
        bge s0, 10, carry_on

        move a0, s0
        li v0, 1
        syscall

        addi s0, s0, 1

        j loop_top
carry_on:
        # carry on
```

The other way around

```
        li s0, 0
loop_top:
        blt s0, 10, loop_code
        b carry_on
loop_code:
        move a0, s0
        li v0, 1
        syscall
        addi s0, s0, 1
        j loop_top
carry_on:
        # carry on
```

While looks the same, no initialization or increment

```
while(s2 < 10)
{
   // stuff!!
}
// more stuff
```

```
loop_top:
    bge s2, 10, more_stuff
stuff:
    # stuff!!
    j loop_top
more_stuff:
    # more stuff
```